

EX1

POST New Request × POST New Request × GET New Request × +

POST http://localhost:8080/api/contacts Send 200 OK 326 ms 98 B

Params Body Auth Headers (4) Scripts Docs Preview Headers (3) Cookies Tests

JSON

```

1 {
2   "nome": "contato 2 teste",
3   "telefone": "9999-9999",
4   "email": "joao@email.com"
5 }
6
7
8

```

Preview

```

1 {
2   "id": 1,
3   "nome": "contato 2 teste",
4   "telefone": "9999-9999",
5   "email": "joao@email.com",
6   "addresses": null
7 }

```

POST New Request × POST New Request × GET New Request × +

GET http://localhost:8080/api/contacts/search?name=thiago Send 200 OK 33 ms 89 B

Params Body Auth Headers (3) Scripts Docs Preview Headers (3) Cookies Tests 0/0

No Body

Preview

```

1 [
2   {
3     "id": 2,
4     "nome": "thiago",
5     "telefone": "9999-9999",
6     "email": "joao@email.com",
7     "addresses": []
8   }
9 ]

```

EX2

POST New Request × PATCH New Request × GET New Request × +

PATCH http://localhost:8080/api/contacts/2 Send 200 OK 38 ms 92 B

Params Body Auth Headers (4) Scripts Docs Preview Headers (3) Cookies Tests 0/0

JSON

```

1 {
2   "email": "thimoraes@gmail.com"
3 }

```

Preview

```

1 {
2   "id": 2,
3   "nome": "thiago",
4   "telefone": "9999-9999",
5   "email": "thimoraes@gmail.com",
6   "addresses": []
7 }

```

Questionario

1- A diferença está em que o SOAP é um protocolo oficial e rigoroso, que exige o uso exclusivo do XML para troca de mensagens e requer um contrato (WSDL) para definir como a comunicação vai ocorrer

O REST é um estilo de arquitetura, que utiliza padrões já existentes como os métodos: GET, POST, PUT E DELETE e permite dados em vários formatos, sendo o principal em JSON.

2

O SOAP ainda é muito utilizado em sistemas legados, como sistemas bancários, telecom e governos.

3

Vantagens do REST

Mais leve e rápido, menor barreira de aprendizado e flexibilidade

Desvantagens

Não tem tanta padronização, segurança terceirizada e não tem estado(Stateless)

4

O WS-Security é uma extensão embutida no SOAP, a vantagem dele é oferecer segurança a nível da mensagem, isso significa que ele consegue encriptar apenas uma parte específica do XML, por exemplo o dado do cartão mas o nome estar visível.

No REST a segurança está no nível de transporte, criando um túnel entre cliente e servidor

5

O Modelo de Maturidade de Richardson é uma forma de avaliar a qualidade de uma API e perceber até que ponto ela segue verdadeiramente o estilo REST. É dividido em 4 níveis (do 0 ao 3):

- **Nível 0 (O Pântano do POX):** Usa o HTTP apenas como um túnel para enviar dados. Geralmente usa-se apenas um URL (Endpoint) para tudo e apenas o método POST.
- **Nível 1 (Recursos):** A API passa a usar múltiplos URLs (ex: `/usuarios`, `/produtos`), mas ainda faz tudo (criar, ler, apagar) usando apenas um método (como o POST ou GET).
- **Nível 2 (Verbos HTTP):** A API começa a usar os métodos de forma correta (GET para ler, POST para criar, DELETE para apagar) e devolve os códigos de estado HTTP corretos (ex: 200 OK, 404 Not Found, 201 Created). *A nossa API dos contactos está neste nível!*
- **Nível 3 (HATEOAS):** O nível perfeito. A API, além de devolver os dados, devolve *links* indicando o que o utilizador pode fazer a seguir (ex: ao pesquisar um contacto, a API devolve os dados e um link do tipo `"href": "/api/contacts/5/addresses"` para saberes onde encontrar os endereços).

Onde o SOAP se encaixa? O SOAP encaixa-se perfeitamente no **Nível 0**. Ele tem geralmente um único endereço de destino (URL) e usa quase exclusivamente o método HTTP POST para enviar um grande "envelope" XML que contém a ação que quer realizar e os dados.

Desafio

Validação

The screenshot shows a REST client interface with a POST request to `http://localhost:8080/api/contacts`. The request body is a JSON object:

```
1 {
2   "nome": "thiago",
3   "telefone": "9999-9999",
4   "email": "joaoemail.com"
5 }
```

The response is a 400 Bad Request with a response time of 32 ms and a body size of 38 B. The response body is a JSON object:

```
1 {
2   "erro": "Formato de e-mail inválido"
3 }
```

POST New Request × PATCH New Request × GET New Request × +

POST http://localhost:8080/api/contacts Send 400 Bad Request 16 ms 39 B Just Now

Params Body Auth Headers (4) Scripts Docs Preview Headers (4) Cookies Tests 0/0 → Mock

JSON

```
1
2 {
3
4   "nome": " ",
5   "telefone": "9999-9999",
6   "email": "joaoe@mail.com"
7 }
8
```

Preview

```
1 {
2   "erro": "O nome não pode estar vazio"
3 }
```

POST http://localhost:8080/api/contacts Send Insomnia / Scratch Pad 20 ms 54 B Just Now

Params Body Auth Headers (4) Scripts Docs Preview Headers (4) Cookies Tests 0/0 → Mock

JSON

```
1
2 {
3
4   "nome": "thiago ",
5   "telefone": "99-9999",
6   "email": "joaoe@mail.com"
7 }
8
```

Preview

```
1 {
2   "erro": "O telefone deve ter entre 8 e 15 caracteres"
3 }
```